# IsabeLLeVM

## Deductive verification of LLVM-IR programs in Isabelle

Research Topics

Author

Thomas Kas
t.s.kas@student.utwente.nl

Supervisors

dr. P. Lammich
E. Putti

# Contents

1 Introduction . . . 3
1.1 Verification Methods . . . 3
1.2 Verification Targets . . . 4
1.3 Verification Back-end . . . 5
1.4 Research Goals . . . 5
1.5 Proposal Contents . . . 5
2 Background . . . 6
2.1 Deductive Verification . . . 6
2.2 LLVM and LLVM-IR . . . 9
2.3 Isabelle . . . 12
3 Preliminary Results . . . 15
3.1 LLVM-IR in Isabelle . . . 15
3.2 Verification Infrastructure . . . 17
3.3 Verification Tools . . . 19
3.4 Verification of Example programs . . . 19
4 Related Work . . . 20
5 Planning . . . 21
5.1 Risk Mitigation . . . 21
Bibliography . . . 23

# 1 Introduction

Software plays a crucial role in many safety-critical areas. It helps steer cars and planes, controls traffic lights and bridges, and synchronizes power plants and electrical grids. With that, its correct functioning is of the utmost importance, as the consequences are great when such software fails. Common software testing methods, for example those involving static or dynamic analysis, only reveal the (potential) presence of bugs, but cannot prove their absence. To assert the absence of bugs, formal verification of the software is required.

A simple example of a program to be verified is shown in Listing 1. The program, written in C, consists of the function `mult`, which takes two parameters `a` and `b`, and should result in an integer of these inputs multiplied together. Its implementation is not quite straightforward as it does not use any multiplication instruction, instead relying on a for-loop with repeated additions. While simple, the program helps illustrate the pros and cons of conventional testing and formal verification.

```c
int mult(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result += a;
    }
    return result;
}
```

Listing 1: C `mult` Function Implementation

## 1.1 Verification Methods

Such programs are commonly tested with dynamic analysis: the program is run for various test-cases with different inputs, and the results are compared to their expected outcomes. These tests are great at quickly identifying major implementation mistakes, however they do not cover all possible inputs and thus cannot assert that the implementation is always correct. Furthermore, they might be analyzed using static analysis tools which help identify common mistakes, but do not know about how the program is supposed to work. Both techniques are useful in their own right, but are not powerful enough to assert a program always works as intended.

As stated previously, formal verification methods are required to assert whether a program is always correct. For example, model-checking can be used to assert correctness for all possible states of an abstract model of the program. While effective, this method is not very efficient. It requires the creation of a model, which itself has to be correct, as well as exploring all possible states of said model, which can quickly become infeasible as the model grows. For our example, this involves trying all possible values for `a` and `b` combined, yielding $2^{64}$ verification conditions.

In contrast, deductive verification does not suffer these shortcomings. It operates on an annotated version of the code of the program and proves correctness using mathematical reasoning. Essentially, it boils down to symbolically executing the program, keeping track of how the execution state and control flow are impacted, and proving that the specified conditions hold for all possible execution states. Note that this does not happen through exhaustively checking all possible execution states as with model checking, but rather using a single symbolic representation that contains all states.

We will create a new deductive verification tool in this thesis, as its formal mathematical basis allows for the most efficient and rigorous verification of the described approaches. To use this method, a target language has to be chosen for which the structure and semantics have to be formally defined. Furthermore, a formal back-end has to be chosen in which these definitions can be created and the deductive verification's proof system can be described.

## 1.2 Verification Targets

Deductive verification can be done at various stages in compilation of software. It might be done with the source code, with fully compiled binaries, or with some intermediary between these stages. Performing verification on source code comes with the advantage that it is relatively easy to integrate the verification into the program, for example with in-line verification instructions, but this comes at the cost of the verification being tied to the specific programming language the software was written in. Targeting a lower level like machine instructions gets around this issue and is the closest to verifying the actual execution of a program, as compilation might introduce artifacts not present in source code, but means that only a single target is verified, tying the verification to a specific architecture. In both cases, changing the language or architecture the verification is done with means porting the verification to a different target as well, introducing more overhead for such tasks.

In this thesis, we will target LLVM-IR. This language lives between a program's source-code and compilation target, being part of the LLVM compilation framework. One if its goals is to allow language-engineers to only target a single compilation representation, LLVM-IR, while supporting many different target architectures, including x86, ARM, and RISC-V. Another goal is to allow developers working on new hardware targets to implement a single back-end for LLVM-IR, automatically enabling support for all languages that compile to it. These goals has been mostly achieved, with some target-specific intrinsics and differences being present in LLVM-IR. Our tool will have to deal with these differences properly when encountered. As such, one does not write programs directly in LLVM-IR, but any supported language can be compiled to it. We can use this to our benefit: once functional, our tool will immediately support software written in a wide range of source languages and targeting many target architectures.

## 1.3 Verification Back-end

Finally, the chosen formal verification back-end for this thesis is Isabelle/HOL. This is an interactive theorem prover that allows users to specify theorems using higher-order logic and mathematically prove them to be correct. Whenever a new theorem is created, it gives the user proof (sub-)goals that have to be proven for the theorem to be accepted. Different tools and solvers are provided to the user which aid in simplifying, breaking up, and eventually proving these goals. Furthermore, it is possible to define custom proof methods for specific tasks using the Eisbach infrastructure. Isabelle is also a functional programming language, allowing users to create their own datatypes and functions, and letting them use these in their lemmas. All of these features combined make Isabelle well-suited to defining the structure and semantics of LLVM-IR, a deductive verification proof-system, and tools that use these formal descriptions, creating an effective deductive verifier for LLVM-IR.

## 1.4 Research Goals

With that, the main research goal of this thesis is:
`RG` - *Creating a deductive verification tool for partial correctness of LLVM-IR programs within Isabelle/HOL.*

This can be achieved through the following sub-goals, producing a minimum viable product:
`SG1` - *Defining the structure and semantics of a targeted subset of LLVM-IR in Isabelle to represent LLVM-IR programs in Isabelle.*
`SG2` - *Implementing Hoare-Floyd style deductive verification infrastructure that works with these semantics.*
`SG3` - *Creating tools to efficiently and effectively verify LLVM-IR programs.*
`SG4` - *Using these tools on selected examples to show their efficacy.*

On top of those main sub-goals, we have several extension goals:
`EG1` - *Extending the defined subset of LLVM-IR to support more of its features, instructions, and intrinsics, such as floating-point values or arrays.*
`EG2.1` - *Creating a tool to port LLVM-IR output into the verifier's representation and vice versa.*
`EG2.2` - *Verifying importing and exporting LLVM-IR is a fully lossless process.*
`EG3` - *Extending the verifier to prove total correctness rather than partial correctness.*
These extension goals are not strictly necessary to achieve the main research goal, but do enhance the functionality of the created tool.

## 1.5 Proposal Contents

Work has already begun implementing the tool described in this proposal. Specifically, goals `SG1`, `SG2`, and `SG3` have been mostly achieved with some room for improvement, described in Section 3. This means the proposal is larger than it would otherwise have been.

Furthermore, Section 2 gives background information on techniques and technologies used in this thesis, Section 4 covers existing literature in this space to take inspiration

from, and finally Section 5 gives a rough planning on how work will proceed from this point forward to complete the proposed tool, as well as some implementation risks and mitigation strategies should they arise.

# 2 Background

This section provides pre-requisite knowledge on the different techniques and tools that lay at the foundation of this thesis. Specifically, Section 2.1 covers how deductive verification works and how it is done, then Section 2.2 explores how LLVM-IR is structured and how it works semantically, and finally Section 2.3 introduces the different features of Isabelle/HOL that will be used to achieve the different goals of this thesis.

## 2.1 Deductive Verification

Deductive verification is a method to mathematically verify that a concrete implementation of a program satisfies specified conditions. There exist many different deductive verifiers for different languages and toolchains, but they share the following foundational principles.

### 2.1.1 Hoare Triples

At the core of deductive verification lie pre-conditions, commands, and post-conditions. The combination of the three, known as Hoare triples, allows one to reason about programs. The pre-conditions, $P$, represent some conditions that have to hold for the command, $c$, to execute properly, while the post-conditions, $Q$, represent the intended result of the command.

```c
/*@
requires a * b < 2^31
requires b >= 0;
ensures \result == a * b;
*/
int mult(int a, int b);
```

Listing 2: C `mult` Function Annotated with ACSL Specifications

Consider Listing 2. It contains a specification for the function `mult`, written in ACSL, a specification language used in deductive verification of C programs using Frama-C. This specifies two pre-conditions and one post-condition for the function, which represents a command. Specifically, the pre-conditions state that the inputs `a` and `b` multiplied should not exceed the signed 32-bit integer limit and `b` should be non-negative for the function to execute properly. If this holds, then the post-condition states that the result will be the inputs `a` and `b` multiplied.

To prove that a post-condition follows from the pre-conditions and the command, the semantic meaning of the command has to be described formally. Consider again the implementation of the `mult` function in Listing 1. To prove anything about this function, the

prover needs to know what exact effects the instructions have on the execution state: how values of the `int` type behave, how assigning variables behaves, how different operations such as ++ and += interact with these values, and how control-flow is impacted by the `for`-loop and `return` keyword.

Once the prover knows the semantics of different instructions, it can assume the pre-conditions hold for the inputs and execute the instructions of a program symbolically, keeping track of how execution state and flow have been impacted thus far. When a return instruction is reached, the post-condition can be checked on the returned value and reached execution state. If it holds for all possible executions, then the program satisfies the specification and is verified deductively. Note that this only proves partial correctness: it might be possible for the program not to terminate, but if it does, then the post-condition holds.

### 2.1.2 Floyd's Method

With that, it is crucial to know the control-flow of all possible executions of a program, as it does not suffice to show correctness for only some executions. For this purpose, Floyd's method can be applied. This involves creating a control-flow graph of the program and creating specifications for all branch-points in said graph. What results is a number of instruction blocks, each annotated with some pre-conditions, and a graph containing the flow between these blocks.

Verification of programs using the annotated instruction blocks and their flow becomes straight-forward. Starting with the first block, its pre-conditions are assumed to hold, and its instructions are executed symbolically. From there, the resulting execution state has to be shown to satisfy the pre-conditions of all possible next instruction blocks. This has to be repeated for all of the annotated blocks. Finally, whenever a block ends by returning, the post-condition of the program has to be shown to hold from the pre-condition and execution of the block. When this is done, the program has been verified to be (partially) correct.

Consider Listing 3. Since the for-loop represents a branch-point in control-flow, some pre-conditions have to be specified for these instruction blocks. For this purpose, loop invariants have been added to the for-loop in the `mult` function. The pre-conditions of the function become the pre-conditions of the initial instruction block, while the loop invariants become the pre-conditions for the loop and final instruction blocks. This is visualized in Figure 1.

```c
/*@
requires a * b < 2^31;
requires b >= 0;
ensures \result == a * b;
*/
int mult(int a, int b) {
    int result = 0;
    /*@
    loop invariant 0 <= i <= b;
    loop invariant result == a * i;
    */
    for (int i = 0; i < b; i++) {
        result += a;
    }
    return result;
}
```

Listing 3: C `mult` Function Implementation Annotated with ACSL Specifications

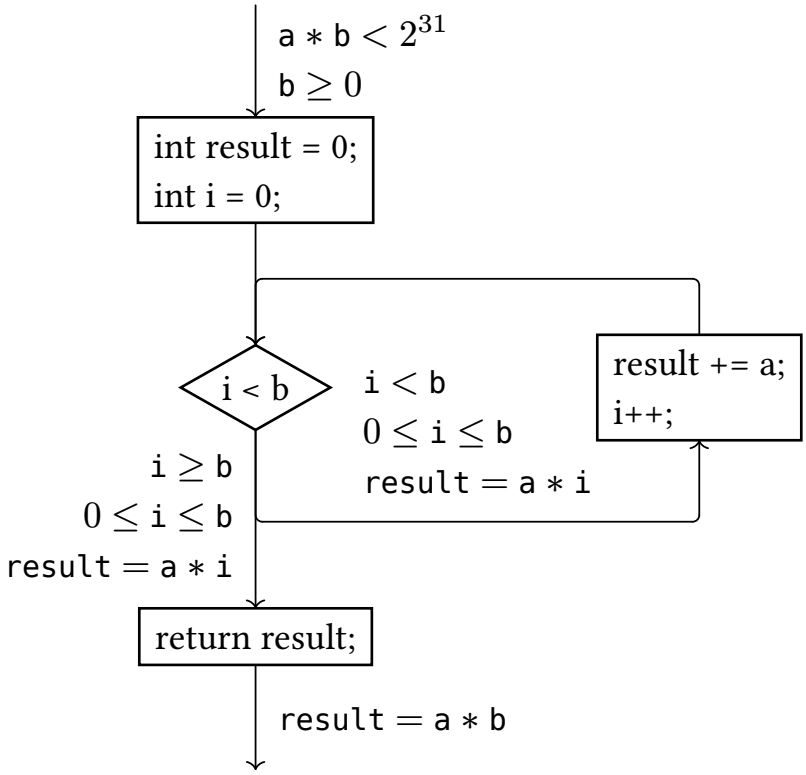


Figure 1: Annotated `mult` Control Flow Graph

With that, the goal of the deductive verification infrastructure is to:

- allow users to specify pre-conditions for a set of instructions with linear control-flow and a general post-condition for functions,
- allow for the symbolic execution of instruction blocks with some assumed conditions, obtaining the execution state,

- allow proving that the pre-conditions of all consequent instruction blocks hold for some obtained execution state, or the post-condition in case the block returned some value.

## 2.2 LLVM and LLVM-IR

LLVM is a toolkit for the construction of compilers, optimizers, and run-time environments. At its core is its own intermediate representation, LLVM-IR, a low-level programming language similar to assembly. It can be used to develop front-ends for any programming language, and back-ends for any target architecture. Currently it supports compiling many source languages, including C, C++, Haskell, and Rust, to a wide range of target architectures, including x86-64, ARM, and RISC-V.

### 2.2.1 LLVM-IR Structure

LLVM-IR is composed at the highest level of different modules, which themselves are made up of functions, global variables, and symbol table entries. The functions consist of an outer definition, including its parameters and return type, and inner basic blocks. These basic blocks are made up of a block label (which are local identifiers), followed by zero or more phi instructions, then some regular instructions, and finally one terminal instruction. Control-flow between basic blocks is determined only by these terminal instructions, as they either specify a local identifier to branch to, or return from the function. The other instructions can only affect the outer control-flow by, for example, calling another function, but not the inner control-flow of the function being executed.

LLVM-IR has two different ways of storing data: in registers, and in memory. Its registers are function-local single-static-assignment values, accessible via identifiers. In other words, they can only be assigned once statically, and their values only remain accessible within the function they are defined in. Their values can however be overwritten when the instruction that assigns them is executed again. Meanwhile, its memory is accessible via specific instructions such as `load` and `store`. These instructions are given the memory addresses to operate on using registers.

Identifiers start with `%` for these local identifiers, and `@` for global identifiers (used for functions and global variables). They have to be unique, either at a local level for local identifiers, or globally for global identifiers. Whenever a value is used as an operand for an instruction, it can either be specified as the identifier of a register, or a constant value.

As an example of a function in LLVM-IR, Listing 4 shows the `mult` function from Listing 1 compiled to LLVM-IR. Its definition specifies its return type is `i32`, and has two `i32` parameters, accessible in registers `%a` and `%b`. It contains 5 different basic blocks: `entry`, `for.cond`, `for.body`, `for.inc`, and `for.end`. None of these blocks contain phi instructions, and are made up of the instructions `alloca`, `store`, `load`, `icmp`, and `add`. They contain the terminal instructions `br` (both the direct `label` and conditional `i1` branch variants) and `ret`. The syntax of these instructions is given in Listing 5.

```llvm
define dso_local noundef i32 @mult(int, int)(i32 noundef %a, i32 noundef %b) {
entry:
  %a.addr = alloca i32, align 4
  %b.addr = alloca i32, align 4
  %result = alloca i32, align 4
  %i = alloca i32, align 4
  store i32 %a, ptr %a.addr, align 4
  store i32 %b, ptr %b.addr, align 4
  store i32 0, ptr %result, align 4
  store i32 0, ptr %i, align 4
  br label %for.cond

for.cond:
  %0 = load i32, ptr %i, align 4
  %1 = load i32, ptr %b.addr, align 4
  %cmp = icmp slt i32 %0, %1
  br i1 %cmp, label %for.body, label %for.end

for.body:
  %2 = load i32, ptr %a.addr, align 4
  %3 = load i32, ptr %result, align 4
  %add = add nsw i32 %3, %2
  store i32 %add, ptr %result, align 4
  br label %for.inc

for.inc:
  %4 = load i32, ptr %i, align 4
  %inc = add nsw i32 %4, 1
  store i32 %inc, ptr %i, align 4
  br label %for.cond

for.end:
  %5 = load i32, ptr %result, align 4
  ret i32 %5
}
```

Listing 4: `mult` Function compiled to LLVM-IR

```llvm
<result> = phi [fast-math-flags] <ty> [ <val0>, <label0>], ...

<result> = alloca [inalloca] <type> [, <ty> <NumElements>] [, align <alignment>] [, addrspace(<num>)]
store [volatile] <ty> <value>, ptr <pointer>[, align <alignment>]
<result> = load [volatile] <ty>, ptr <pointer>[, align <alignment>]
<result> = add [nuw] [nsw] <ty> <op1>, <op2>
<result> = icmp [samesign] <cond> <ty> <op1>, <op2>

br i1 <cond>, label <iftrue>, label <iffalse>
br label <dest>

ret <type> <value>
ret void
```

Listing 5: Syntax of LLVM-IR Instructions

### 2.2.2 LLVM-IR Semantics

When executing functions, its first instruction block (which is the only one where a label is not mandatory) is executed first. From there, the terminal instructions of blocks dictate whether another block is executed, or the function returns. Within blocks, instructions are executed in-order. As this thesis will only encapsulate a sub-set of LLVM-IR instructions, only the semantics of those included will be shown here, which are: `phi`, `alloca`, `store`, `load`, `add`, `icmp`, `br`, and `ret`. As the scope of the thesis increases to cover more instructions, this section will grow to also explain their semantics.

The fact that LLVM-IR functions are already subdivided into basic blocks with straightforward control-flow makes it ideal for graph-based deductive verification. Each basic block can only lead to one other predetermined basic block, or to one of two based on a boolean value, so constructing a control-flow graph is simple. With that, the biggest challenge for the design of the verifier is how to annotate basic blocks with pre-conditions and how to symbolically execute instructions from there, keeping track of register and memory state modifications.

#### 2.2.2.1 Phi Instructions

The `phi` instructions always have to appear at the start of an instruction block. They assign some value from the list `[ <val0>, <label0>], ...` to the register `<result>`, specified by the pair corresponding to the predecessor block that branched to this block. As such, it allows for a conditional register assignment which is a valid SSA-value.

#### 2.2.2.2 Regular Instructions

`alloca` instructions allocate memory on the stack frame of the current function, which is released automatically when the function returns. They yield a pointer to the newly allocated memory, which is stored in the register `<result>`.

`store` instructions store a value (from a register or a constant value) at some memory address from a pointer (e.g. an address space in a register, as return by `alloca`). It does not yield any value.

`load` instructions read a value at a memory address from a pointer. It yields the read value to be stored into the register `<result>`.

`add` instructions add together two values of the same type, `<op1>` and `<op2>` and yields the result to be stored into `<result>`. Depending on whether `nuw`, no unsigned wrap, or `nsw`, no signed wrap, were specified, and either an unsigned or signed overflow occurred respectively, a poison value is produced. Such a poison value propagates whenever it is used in any computations, and leads to undefined behavior when it is used for certain instructions, for example those that affect control-flow and memory addressing.

`icmp` instructions compare two integer values according to the specified comparison operand `<cond>`. This is one of `eq` for equal, `ne` for not equal, `ugt` for unsigned greater than, `uge` for unsigned greater or equal, `ult` for unsigned less than, and `ule` for unsigned less or equal comparisons, or `sgt`, `sge`, `slt`, and `sle` for their signed counterparts. If `samesign` was specified then the operation produces a poison value if the operands are not of the same sign. It yields a single-bit integer (a boolean) for the comparison to be stored into `<result>`.

#### 2.2.2.3 Terminal Instructions

`br label` instructions branch to the basic block with the specified label unconditionally, while `br i1` branches to the block with the first label if the value `<cond>` is true, or the block with the second label if it is false. This has undefined behavior is `<cond>` is a poison value.

`ret` returns from a function with the specified `<value>`, or with no value if the type is `void`.

## 2.3 Isabelle

Isabelle/HOL (referred to as just Isabelle from here) is an interactive theorem prover with higher-order logic features, allowing users to create and prove correct theories consisting of lemmas. These lemmas, representing singular theorems, can contain several assumptions, and define a single conclusion from the assumptions. Whenever a new lemma is defined using the keyword `lemma`, Isabelle prompts the user to prove one or more proof goals. With provided tools, the user is able to simplify, break up, rewrite, and eventually prove these goals (assuming they are correct). Once all goals have been proven, the lemma is added as correct to the theory it is defined in and can be used in more complex proofs.

Isabelle is also a functional, statically-typed language, allowing users to create functions which they can then prove certain properties about in lemmas. Functions can have several

parameters of specific types, and always produce a result of a single type. They can use pattern matching over the parameters to specify different function definitions depending on the input. Simple recursive functions created using the keyword `fun` and are automatically proven to terminate, while functions created using the keyword `function` require the user to prove they terminate. It is possible to define functions that do not terminate with the keyword `partial_function`, which omits the need to prove termination. As an example, the `mult` function has been written in Isabelle as a recursive function over the second parameter in Listing 6. Here, the parameters are natural numbers, which are defined as zero, `0`, or recursively as the successor of another natural number, `Suc n`. If the second parameter is zero, then the result of the function is also zero, but if it is the successor of a natural number, `b`, the function recurses to `mult a b` (`a` being the first parameter) and adds `a` to that.

```isabelle
fun mult where
  "mult a 0 = 0"
| "mult a (Suc b) = a + mult a b"
```

Listing 6: Recursive Function `mult` in Isabelle

Proving that the `mult` function indeed multiplies the two inputs is possible by creating a lemma that states `mult a b` is equal to `a * b`, and proving the resulting proof goals. This has been done in Listing 7. The proof works as follows:

- applying induction over `b` using `apply (induction b)`, to get two new sub-goals:
  - the base case: `mult a 0 = a * 0`
  - the inductive step: $\bigwedge$`b. mult a b = a * b` $\Longrightarrow$ `mult a (Suc b) = a * (Suc b)`
- applying the simplifier to the base case using `apply simp`, which internally does the following to the sub-goal:
  - rewrite `mult a 0` to its definition: `0`,
  - rewrite `a * 0` to `0` using an internal definition of multiplication of natural numbers,
  - prove the resulting sub-goal `0 = 0` by reflection.
- applying the simplifier to the inductive step using `apply simp`, which does the following:
  - rewrite `mult a (Suc b)` to its definition `a + mult a b`,
  - use the assumption (which is specified to hold for any b) to rewrite `mult a b` to `a * b`,
  - rewrite `a * (Suc b)` to `a + a * b` using an internal definition of multiplication of natural numbers,
  - prove the resulting sub-goal `a + a * b = a + a * b` by reflection.
- the lemma is added to the theorem with `done`, which can only be done when no sub-goals are left over.

```isabelle
lemma "mult a b = a * b"
  apply (induction b)
  apply simp
  apply simp
  done
```

Listing 7: Lemma for `mult` Function

Moreover, Isabelle allows user to create custom datatypes to be used in functions and lemmas using the keyword `datatype`. These datatypes can have any number of uniquely-named constructors which can consist of any number of values of different types. It is also possible for datatypes to contain generic types, which can be instantiated with some type whenever they are used. Examples of a custom natural number and list datatype are shown in Listing 8. Here, `custom_nat` is defined as either `custom_zero`, representing zero, or `custom_succ custom_nat`, representing the successor of another `custom_nat`. The value shown below it represents the natural number `2`. Furtermore, `custom_list` uses a generic type `'a`, which is defined as either `custom_nil`, representing an empty list, or `custom_cons`, representing a value of `'a` prepended to another `custom_list` instantiated with the same generic type. The value shown below it represents the list with `[0, 1]`.

```isabelle
datatype custom_nat = custom_zero | custom_succ "custom_nat"
value "custom_succ (custom_succ custom_zero)"

datatype 'a custom_list = custom_nil | custom_cons "'a" "'a custom_list"
value "custom_cons custom_zero (custom_cons (custom_succ custom_zero) custom_nil)"
```

Listing 8: Isabelle Datatype Examples

As previously mentioned, Isabelle provides different reasoning tools to simplify, break up, and prove sub-goals. One such example is the Simplifier, as invoked in Listing 7, which supports higher-order rewriting. Additional tools include the classical reasoner, classical tableau prover, Metis prover, and Sledghammer. Users can use such tools, but also directly apply rules to rewrite or prove goals if their conditions for application are met. For example, the rule `refl` can prove sub-goals by reflection if the goal is of the form `x = x` for some `x`.

Moreover, users can define their own proof methods using Eisbach, Isabelle's built-in proof method language. With Eisbach, invoked with the `method` keyword, it is possible to combine different rules and reasoning tools for specific use-cases. Such user-defined methods can be applied the same way as other pre-defined proof methods, using the `apply` keyword followed by the specified method name. For example, the rule `asm_rl` can be applied, which only succeeds for sub-goals of a specific form, followed by another proof method capable of solving or simplifying that specific sub-goal. It also features built-in

sub-goal pattern matching for this end, which is more powerful as it then allows using variables from matched sub-goal for proof methods.

The first technique to apply certain proof-methods for specific subgoals is used in Listing 9, where a similar proof as Listing 7 is given. Here, the method `example_base` can only be used for the base step of the induction proof, as applying `rule asm_rl[of "mult _ 0 = _"]` only succeeds when a sub-goal of that form is present. Similarly, the method `example_inductive` can only be used for the inductive step, as it needs a sub-goal where the second parameter of `mult` is a successor of another natural number. Trying to apply the methods to other sub-goals fails.

```
method example_base = rule asm_rl[of "mult _ 0 = _"], simp
method example_inductive = rule asm_rl[of "mult _ (Suc _) = _"], simp

lemma "mult a b = a * b"
  apply (induction b)
  apply example_base
  apply example_inductive
  done
```

Listing 9: Custom Proof Methods in `mult` Lemma

# 3 Preliminary Results

This section covers the preliminary results from work already done. Specifically, it details how `SG1` has been achieved in Section 3.1,how much has been achieved for `SG2` in Section 3.2, and how `SG3` and `SG4` will be tackled in Section 3.3 and Section 3.4.

## 3.1 LLVM-IR in Isabelle

A subset of LLVM-IR's structure and semantics have been defined in Isabelle. The chosen subset covers basic language features that allow simple programs, such as the `mult` function from Listing 4, to be described and executed. Specifically, the structure of some instructions, basic blocks, functions, and programs have been defined, as well as their semantics.

The structure has been defined as a deep embedding, using datatypes to represent its AST. This means that representing LLVM-IR using our representation requires a conversion, which a shallow embedding would not necessarily have required. However, this comes with the benefit of being more flexible in terms of what can be done with the embedded programs. A shallow embedding would have been more limited in the ways we could analyze, execute, and reason about embedded programs, which would have relied on built-in Isabelle features.

For example, a datatype `llvm_instruction` has been defined which can be instantiated as any of the supported instructions with their specific options according to their definition in LLVM's language reference manual. Its definition is shown in Listing 10. Such definitions

have been made for types, typed values, identifiers, value references (either a direct value, or value from a register), alignment specifications, add wrap options, compare conditions, phi instructions, regular instructions, terminator instructions, basic blocks, block return values, functions, and programs.

```isabelle
datatype llvm_instruction = alloca llvm_register_name llvm_type "llvm_align option"
                          | store llvm_type llvm_value_ref llvm_pointer "llvm_align option"
                          | load llvm_register_name llvm_type llvm_pointer "llvm_align option"
                          | add llvm_register_name llvm_add_wrap llvm_type llvm_value_ref llvm_value_ref
                          | icmp llvm_register_name llvm_same_sign llvm_compare_condition llvm_type llvm_value_ref llvm_value_ref
```

Listing 10: Datatype Definition for LLVM-IR Instructions

On top of this representation of the AST, the semantics are defined using functions that form an interpreter. It is made up of functions implementing different execution levels, each being made up of the functions of the level below it:

- state operations: reading from and writing to memory and registers,
- instructions: performing state operations and applying some additional logic,
- basic blocks: consecutively executing instructions, its definition is shown in Listing 11,
- functions: executing basic blocks and handling control-flow. Note that this execution function is partial, which means it does not guarantee termination. It is possible for functions to loop forever.

At the core of this interpreter lies the `state`, a triple of registers, stack memory, and heap memory. State operations directly interact with its internal definition, providing abstractions that make the easier to operate on and reason about.

Furthermore, it uses the `result` datatype, which can either represent successful execution containing some value, or erroneous execution containing an error message. Monad laws have been defined on top of it, making it easy to chain together function calls that might produce an error. This monad short-circuits execution when an error is encountered, or flows naturally when execution is successful. An error is produced when some execution requirement of an instruction is not met, such as when the `load` instruction tries to load a value from uninitialized memory.

Together, the defined structure and semantics can be used to execute functions made up of the supported instructions.

```isabelle
fun execute_block :: "llvm_label option ⇒ llvm_instruction_block ⇒ state ⇒ (state * llvm_block_return) result" where
  "execute_block previous ((phi name type values)#ps, is, t) s = do {
    s' ← execute_phi previous name values s;
    execute_block previous (ps, is, t) s'
  }"
| "execute_block previous ([], i#is, t) s = do {
    s' ← execute_instruction i s;
    execute_block previous ([], is, t) s'
  }"
| "execute_block previous (_, _, br_i1 v l1 l2) s = do {
    val ← get_register s v;
    label ← (case val of (vi1 b) ⇒ ok (if b then l1 else l2) | _ ⇒ err incompatible_types);
    return (s, branch_label label)
  }"
| "execute_block previous (_, _, br_label l) s = return (s, branch_label l)"
| "execute_block previous (_, _, ret t v) s = do {
    res ← get_register s v;
    return (s, return_value res)
  }"
```

Listing 11: Basic Block Execution Function

## 3.2 Verification Infrastructure

Building on the defined semantics, we have already implemented verification infrastructure to achieve most of SG2. At its core are weakest pre-conditions defined for the `result` type, shown in Listing 12. These can be used in lemmas to prove that given some pre-conditions, execution of some command `c` will always be successful, and some post-conditions hold for the return value of `c`.

```isabelle
definition wp :: "'a result ⇒ ('a ⇒ bool) ⇒ bool" where
  "wp c Q = (case c of ok v ⇒ Q v | err _ ⇒ False)"
```

Listing 12: `wp` Definition

These conditions are specified using abstractions over the state for easy verification, `memory_`$\alpha$ and `register_`$\alpha$. These abstractions can be used to assert whether some memory address is allocated or initialized or not, whether it contains a specific value, and whether specific registers contain any or a specific value.

For example, consider the `load` instruction. It reads a value stored in memory from a pointer stored in a register into another register. As such, to execute successfully, it needs the given register to contain a pointer, and corresponding memory address to be allocated and initialized. Furthermore, when executed successfully, it leaves memory exactly as it was before, but updates a single register to the value read from memory. These facts have been described using our weakest precondition definition in Listing 13. Here, the assumptions of the lemma are the pre-conditions, the first argument for `wp` is the command, and the second argument is the post-condition.

```isabelle
lemma
  assumes "register_α s pointer = Some (addr a)"
  and "memory_α s a = Some (Some v)"
  shows "wp
    (execute_load name pointer s)
    (λs'. memory_α s' = memory_α s
    ∧ register_α s' = (register_α s)(reg name := Some v))"
```

Listing 13: `load` Weakest Precondition Lemma

Such lemmas have been defined from the bottom up for all state operations, instructions, and basic blocks. By combining these lemmas with a consequence rule it is possible to use them in proofs for higher-level weakest preconditions, yielding new sub-goals for each of the preconditions of the constituent commands.

For example, `execute_load` is made up of the state operations `get_register`, `get_memory`, and `set_register`, along with some logic to ensure `get_register` returned an address. Applying their respective weakest pre-condition lemmas combined with the consequence rule, yields the following sub-goals for the lemma in Listing 13:

- `get_register`'s precondition:
  `register_α s pointer ≠ None`
- `get_register`'s postcondition and proving its value is an address:
  `register_α s pointer = Some x ⟹ ∄xa. x = addr xa ⟹ False`
- `get_memory`'s precondition following from `get_register`'s postcondition:
  `register_α s pointer = Some x ⟹ x = addr xa ⟹ memory_α s xa ≠ None`
- `get_memory`'s precondition:
  `register_α s pointer = Some x ⟹ x = addr xa ⟹ memory_α s xa ≠ Some None`- `register_α s pointer = Some x ⟹ x = addr xa ⟹ memory_α s xa = Some (Some xb) ⟹ register_α xc = (register_α s)(reg name ↦ xb) ∧ memory_α xc = memory_α s ⟹ memory_α xc = memory_α s ∧ register_α xc = (register_α s)(reg name ↦ v)`
  (`get_register` and `get_memory`'s postconditions, with `execute_load`'s postcondition)

All of these can be proved to hold given the assumptions, so `execute_load` has been proven to always execute correctly given these assumptions. Moreover, given successful execution, the postcondition has also been proven to hold for the returned state.

The workings of these weakest preconditions are essentially the same as that of Hoare triples. The connection becomes clear with our weakest preconditions expressed as `P ⟹ wp c Q`, representing the Hoare triple `P c Q`. Because most literature is based on Hoare triples and being simpler to define, we have created a simple definition to create Hoare triples in terms of weakest preconditions. This unfolds `hoare P c Q` to the form `P s ⟹ wp (c s) Q`, as shown in Listing 14. What this means intuitively, is that we assert that for all states that satisfy the precondition `P`, applying the command `c` to that state yields a new state that satisfies the postcondition `Q`.

```
definition "hoare P c Q ≡ (∀s. P x → wp (c s) Q)"
```

Listing 14: Hoare Triple Definition

With that, Hoare triples can be defined for the execution of basic blocks. Using our weakest precondition rules, we can obtain sub-goals to prove that the basic block executes successfully, as well as an abstract representation of the state reached after the block has been executed.

On top of that, we have defined methods to extend this verification to entire functions based on Floyd's method, including creating a control-flow graph. However, these remain unfinished, so we will need to finish these going forward to fully achieve `SG2`.

## 3.3 Verification Tools

Work has begun creating verification tools to use our Hoare triples intuitively and effectively. For example, Eisbach methods have been created to automatically prove specific sub-goals for different instructions as well as handle multi-block proofs. These are already effective at clearing sub-goals of the instructions within a block, leaving the user to deal with just the final proof goal, or any failing sub-goals which indicate missing pre-conditions. While mostly functional, these remain incomplete and depend on the completion of `SG2` to finish, which means there is still some work to do to achieve `SG3`.

## 3.4 Verification of Example programs

Finally, some simple example programs have been encoded into our LLVM-IR structure. These have not been fully verified as we need `SG2` and `SG3` to be completed before that, but their basic blocks have been useful in testing the functionality of what has been implemented already. One such program is the `mult` function from Listing 4. Of course, more programs of similar or higher complexity need to be defined and verified before `SG4` can be considered achieved.

# 4 Related Work

There exist quite a few deductive verifiers for specific purposes, toolchains, and languages. Many of them only support a single language or language family. For example, Frama-C focuses on the C language, with its WP plugin providing deductive verification of C programs based on weakest-precondition calculus just like our verifier.[1]

Others, like Viper, take an approach similar in philosophy to LLVM, where a common verification backend is established for which different frontends can be created to support new languages.[2] Viper defines its own intermediate language which it is able to verify, making it possible to compile other languages to this language to verify them. Its verification is based on verification condition generation or symbolic execution depending on the user's preferences.

Alongside first-party Viper frontends for specific languages like Python, Rust, and Go, there is also the VerCors frontend.[3] VerCors is a deductive verification tool made specifically for concurrent and parallel programs, supporting a variety of source languages like Java and OpenCL. Interestingly, there exists a project within VerCors, PALLAS, to support the verification of LLVM-IR.[4] However, this project is still being worked on and not ready to be used.

Dafny flips this logic, instead providing a source language to create and verify programs in, which can then be compiled to different programming languages.[5] It uses Boogie internally for verification, which itself comes with another intermediate verification language. While this approach works well for new programs that are verified from the start, this does not address the need to verify existing programs.

Like Viper, $\mathbb{K}$ is a semantic framework that aims to solve the issue of creating a verification tool for each programming language.[6] It uses the deductive proof assistant KIV which is also based on weakest precondition calculus.[7] $\mathbb{K}$ allows describing the syntax and semantics of languages, from which algebraic specifications to be used in KIV can be derived.

All previously mentioned verifiers mainly focus on verifying programs using their source code. Of course, there are also cases where a program's source code is no longer available, but its binary needs to be verified. There exist different verifiers for these purposes, such as for memory preservation of x86-64 binaries based on symbolic execution and Floyd-style verification.[8] A somewhat related verifier has also been created to check source-level specifications within the produced x86 binary, based also on Floyd's method.[9]

In conclusion, most if not all existing deductive verification tools involve similar methods as used in this thesis: weakest precondition calculus, Hoare logic, Floyd-style verification, and/or verification condition generation. Additionally, many of them aim to tackle the same problem: how to get around the need to define new verifiers for each programming language. This thesis differs from most literature in the way it aims to solve this problem, by creating a verifier for LLVM-IR, which most programming languages can be compiled

to. Furthermore, due to its low-level nature, it is relatively easy to decompile program binaries into LLVM-IR. While other initiatives for this purpose exists, they are not yet in a production-ready state.

# 5 Planning

Much of the minimum viable product has already been implemented. The remaining work for the MVP will be implemented according to the GANTT chart shown in Figure 2. Furthermore, it shows the extra features to be implemented on top of the MVP, along with rough estimates on how long they will take to complete and when they might be implemented. In addition to that, it shows roughly when work on the tool itself will conclude, and the accumulated work and research will be gathered and written into the final thesis document. The current aimed-for date to complete the thesis is the 19th of June, which can be extended if necessary.

## 5.1 Risk Mitigation

With any project, especially of the size of a thesis, risks are present that might endanger its feasibility and completion. For the completion of this project, running out of time before implementing all proposed features is the biggest risk due to uncertain development time and complexity. For this reason, a minimum viable product was defined (`SG1-4`) which represents the core goal of the research: creating a deductive verifier for a subset of LLVM-IR in Isabelle. When this core goal is achieved, additional features (`EG1-4`) can be implemented that are not strictly necessary for the functionality of the tool, but do extend its usefulness. Given that a large part of the MVP has already been implemented, there is only a low risk that it cannot be completed before the deadline. The extension goals have been formulated in an open-ended way so their scope can scale to the time left for their implementation, or some can be left out entirely if others are deemed more important. Currently, `EG2` and `EG3` are deemed most important for the usefulness of the final tool, so the initial time after completing `SG1-4` will be spent on them.

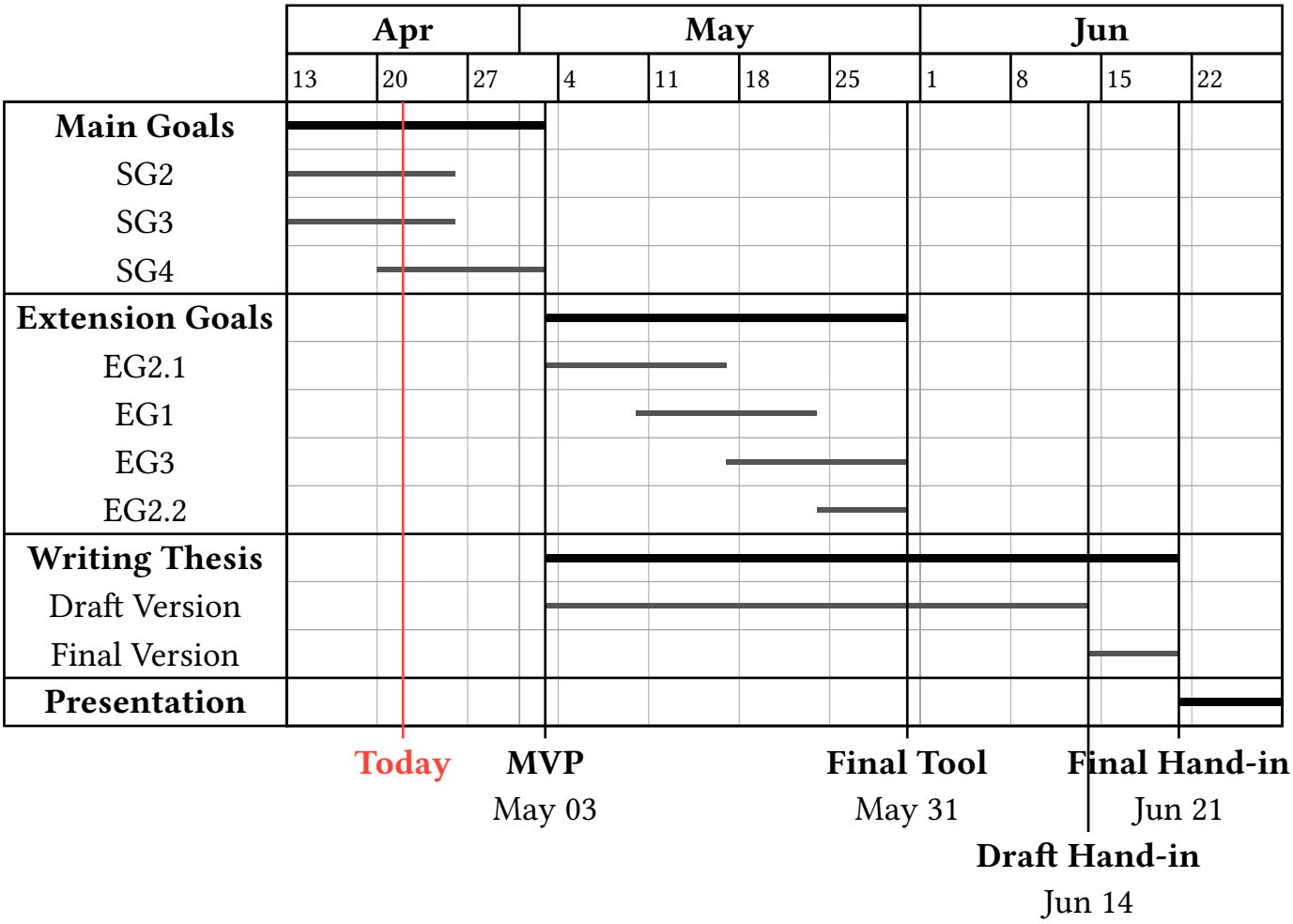


Figure 2: GANTT Chart of Project Planning

# Bibliography


[1] A. Blanchard, F. Loulergue, and N. Kosmatov, "Towards Full Proof Automation in Frama-C Using Auto-active Verification," in *NASA Formal Methods*, Springer International Publishing, 2019, pp. 88–105. doi: 10.1007/978-3-030-20652-9_6.

[2] P. Müller, M. Schwerhoff, and A. J. Summers, "Viper: A Verification Infrastructure for Permission-Based Reasoning," in *Verification, Model Checking, and Abstract Interpretation*, Springer Berlin Heidelberg, 2015, pp. 41–62. doi: 10.1007/978-3-662-49122-5_2.

[3] S. Blom, S. Darabi, M. Huisman, and W. Oortwijn, "The VerCors Tool Set: Verification of Parallel and Concurrent Software," in *Integrated Formal Methods*, Springer International Publishing, 2017, pp. 102–110. doi: 10.1007/978-3-319-66845-1_7.

[4] D. van Oorschot, M. Huisman, and Ö. Şakar, "First Steps towards Deductive Verification of LLVM IR," in *Fundamental Approaches to Software Engineering*, Springer Nature Switzerland, 2024, pp. 290–303. doi: 10.1007/978-3-031-57259-3_15.

[5] K. R. M. Leino, "Dafny: An Automatic Program Verifier for Functional Correctness," in *Logic for Programming, Artificial Intelligence, and Reasoning*, Springer Berlin Heidelberg, 2010, pp. 348–370. doi: 10.1007/978-3-642-17511-4_20.

[6] D. Klumpp and P. Lenzen, "\mathbb K and KIV: Towards Deductive Verification for Arbitrary Programming Languages", in *Recent Trends in Algebraic Development Techniques*, Springer International Publishing, 2021, pp. 98–119. doi: 10.1007/978-3-030-73785-6_6.

[7] G. Ernst, J. Pfähler, G. Schellhorn, D. Haneberg, and W. Reif, "KIV: overview and VerifyThis competition," *International Journal on Software Tools for Technology Transfer*, vol. 17, no. 6, pp. 677–694, Apr. 2014, doi: 10.1007/s10009-014-0308-3.

[8] J. A. Bockenek, F. Verbeek, P. Lammich, and B. Ravindran, "Formal Verification of Memory Preservation of x86-64 Binaries," in *38th International Conference on Computer Safety, Reliability and Security*, Sept. 2019.

[9] A. Kamkin, A. Khoroshilov, A. Kotsynyak, and P. Putro, "Deductive Binary Code Verification Against Source-Code-Level Specifications," in *Tests and Proofs*, Springer International Publishing, 2020, pp. 43–58. doi: 10.1007/978-3-030-50995-8_3.